

Vector Embedding

❖ Summary: -

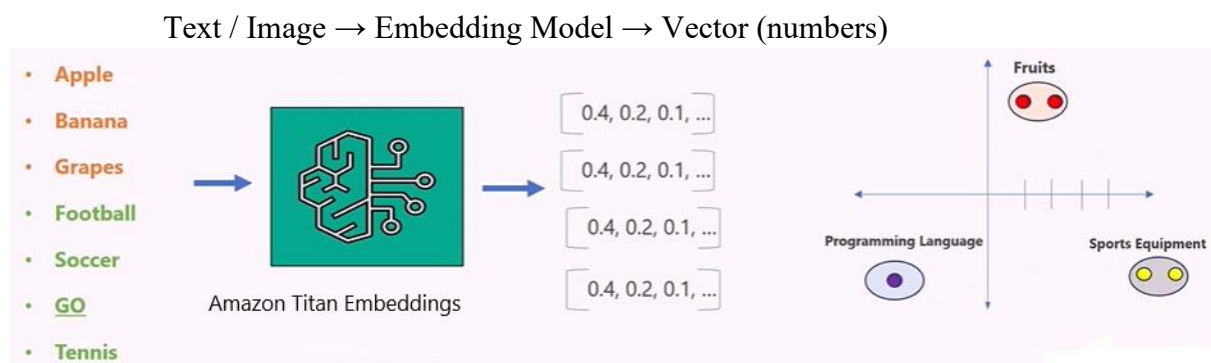
I created a system using Amazon Amazon Bedrock and a Lambda function that takes user text, converts it into vector data using an AI model, and returns the result. I also understood concepts like vectors, embeddings, data chunking, vector storage, and vector search used in AI applications.

❖ Vectors: -

- Why Do We Need Vectors: -
 - Computers don't understand text, images, or audio directly. They understand only numbers.
 - So we convert everything into vectors (arrays of numbers).
- What are Vectors: -
 - Vectors are mathematical representation of words, sentences and documents.
 - Vectors are represented by list of numbers or sequence of numbers.

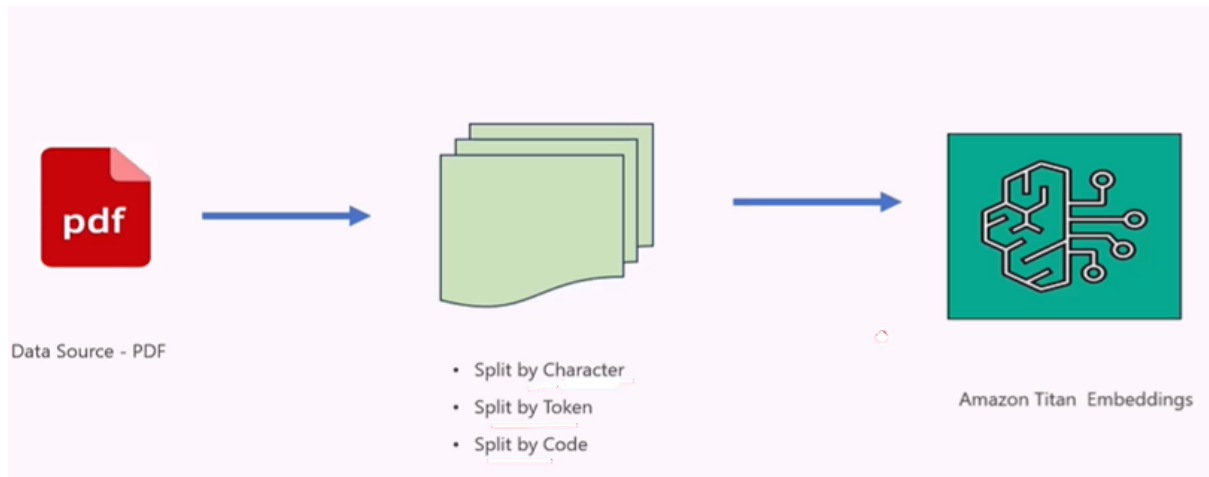
❖ Embedding Models: -

An embedding model is an AI model that transforms data into numerical vectors so that machines can understand and compare meaning.



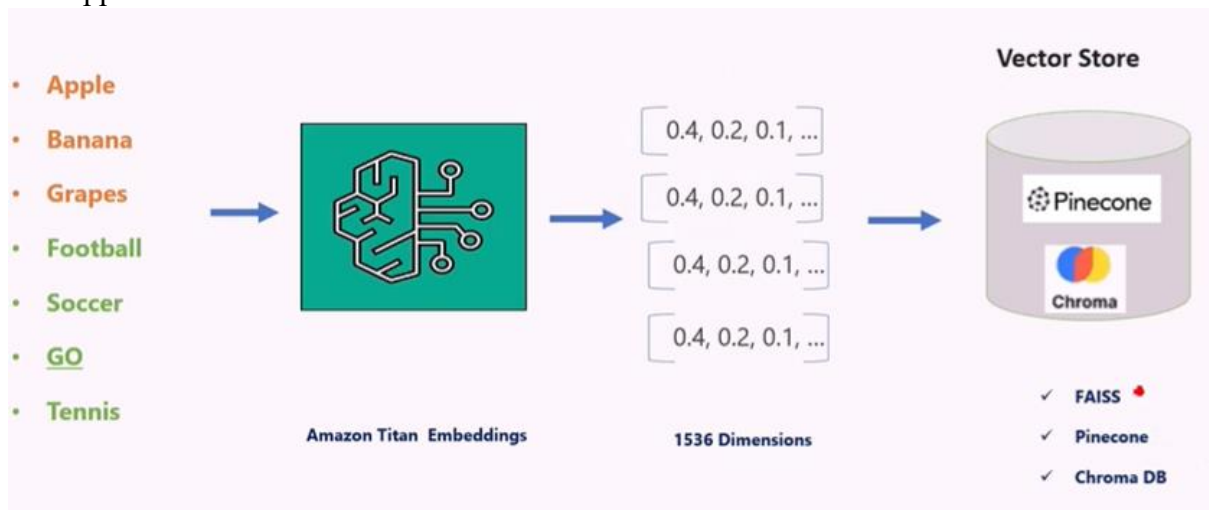
❖ Data Chunking: -

Data chunking is the process of dividing data into manageable pieces (chunks) so AI can process it efficiently.



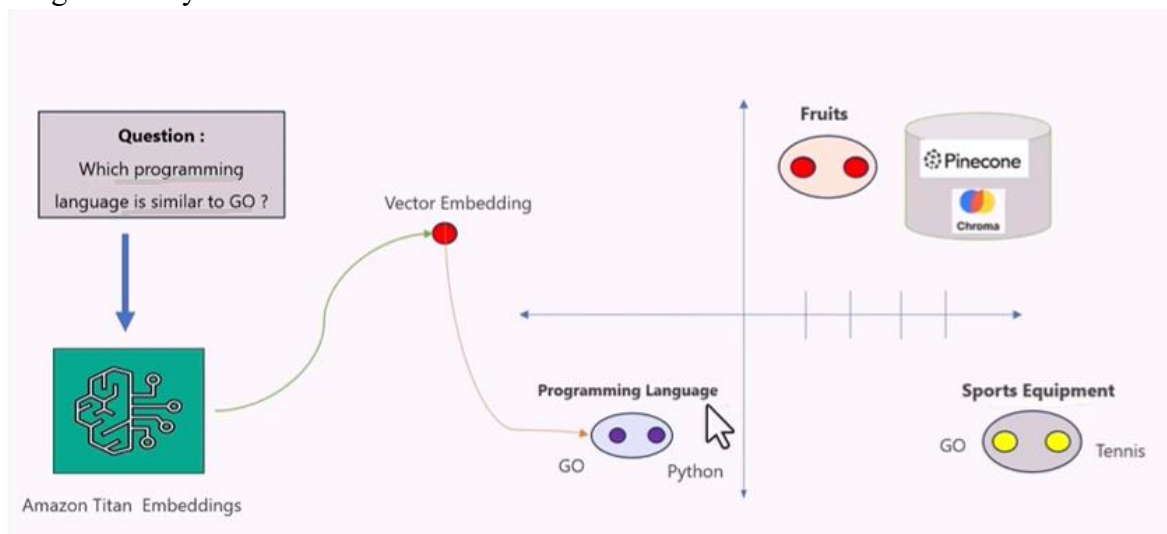
❖ Vectors Store: -

A vector store is a database that stores embedding's and enables fast similarity search for AI applications.



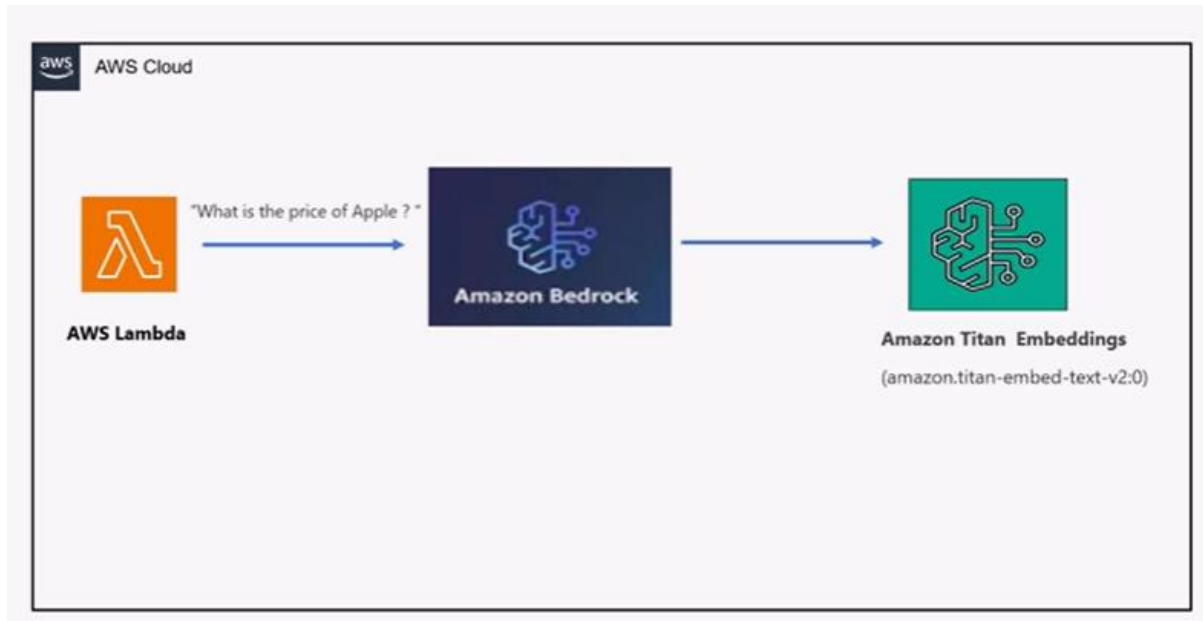
❖ Vector Search: -

Vector search is the process of finding similar items by comparing their vector representations using similarity measures.



❖ Vector and Embedding's In AWS Cloud: -

- So what we will do we have the AWS Cloud.



- With AWS cloud we create Lambda function.
- And through the lambda function we are going to send some prompts (What is the price of Apple?).
- Then we are going to send this word of sentences to Amazon Bedrock
- And Amazon bedrock behind the scenes is going to send this request to Amazon Titan Embedding's.
- And in particular we are going to use Amazon Dot titan Embed text version 2.0 Model.
- So once the request is going to Amazon titan embedding model.
- Its going to create a vector embedding of that particular model and send back the response to the lambda function.

❖ Now we are going to build this: -

- Firstly go to your console and search Amazon Bedrock.
- Then go in view model catalog
- And search Titan Text Embeddings V2 in search bar.

Model catalog (256)

Discover Bedrock serverless or Marketplace models that best fit your use case. To get started using a serverless model, select it from the model catalog and open it in the playground. For Marketplace models, subscribe and deploy.

Serverless (104) | **Bedrock Marketplace (152)**

Filters

▼ **Providers**

- ☐ AI/21 Labs (0)
- ☐ Amazon (0)
- ☐ Anthropic (0)
- ☐ Cohere (0)
- ☐ DeepSeek (0)
- ☐ Google (0)
- ☐ Meta (0)
- ☐ MiniMax (0)
- ☐ Mistral AI (0)
- ☐ Moonshot AI (0)

[Show 10 more](#)

► **Spotlight**

Find models 4 matches

Model name	Version	Provider name	Popularity	Deployments	Deployment type
Titan Image Generator G1	v2	Amazon	34	N/A	Serverless
Titan Text Embeddings V2	v2.0	Amazon	35	N/A	Serverless
Titan Multimodal Embeddings G1	v1	Amazon	36	N/A	Serverless
Titan Embeddings G1 - Text	v1.2	Amazon	37	N/A	Serverless

- Open this [Titan Text Embeddings V2](#)

Introducing the AWS Serverless generative AI webinar series
Join AWS Serverless experts for a hands-on workshops on Agentic AI, Intelligent Document Processing, and building real-time APIs for chatbots. [See more details](#)

Functions (3) Last fetched 14/04/2026, 12:49:32 [Actions](#) [Create function](#)

Search by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	writetoElastic	-	Zip	Python 3.11	Standard	3 months ago
☐	test1	-	Zip	Python 3.11	Standard	3 months ago
☐	readValuefromParas	-	Zip	Python 3.11	Standard	4 months ago

- Now open Another AWS console in new tab and Search Lambda in search bar.
- Click on the create function.

Create function

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Durable execution - new
Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes. [View pricing](#)
☐ Enable

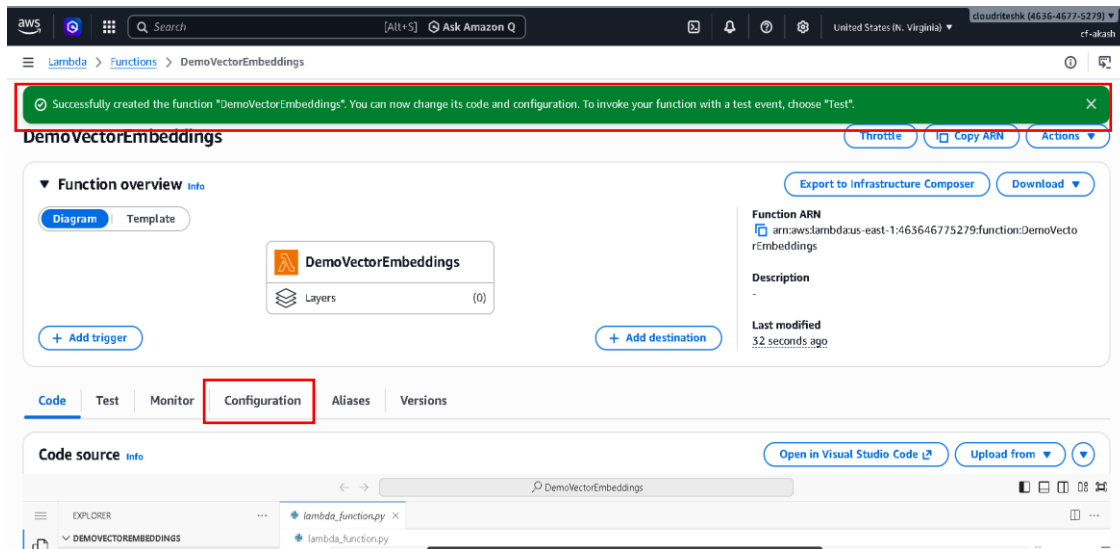
Architecture
Choose the instruction set architecture you want for your function code.
☐ arm64
☒ x86_64

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.
[Change default execution role](#)

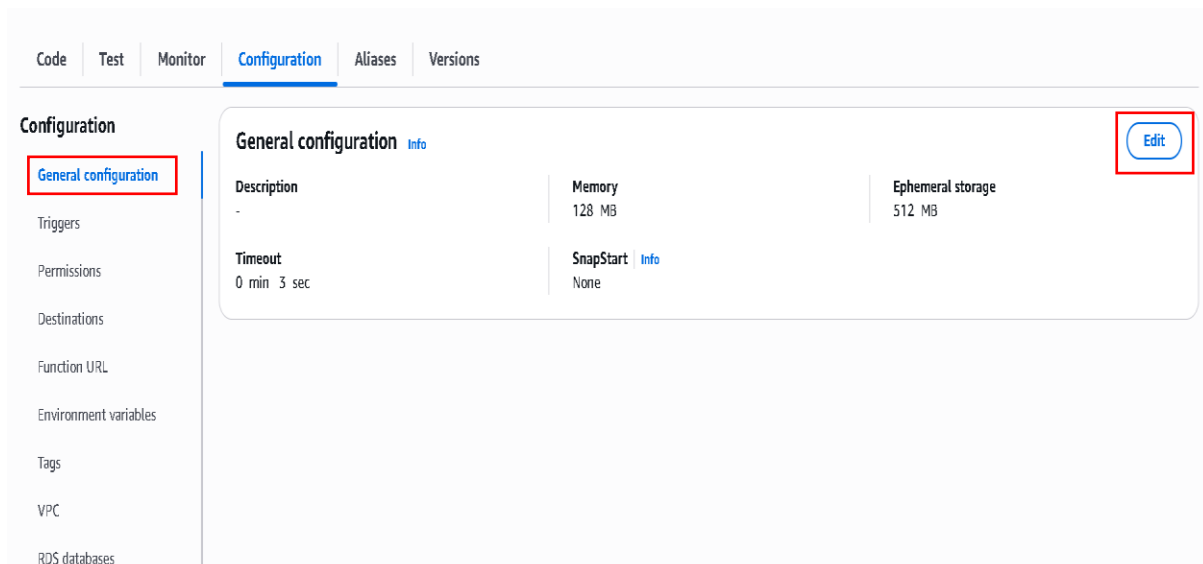
Additional configurations
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

[Cancel](#) [Create function](#)

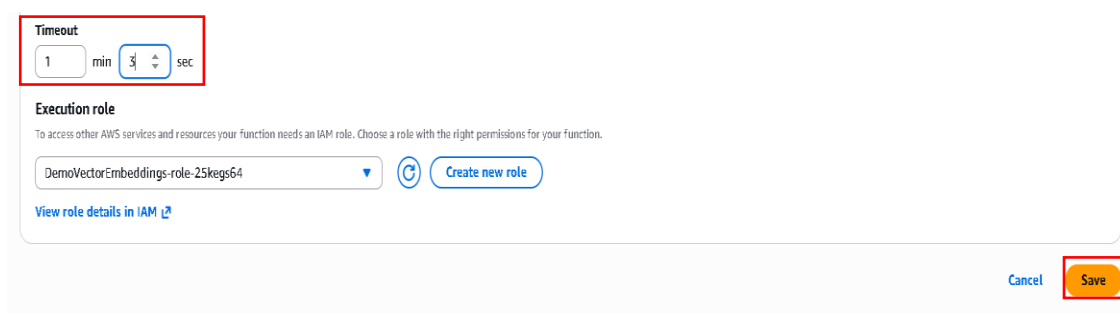
- Now click on create Account.



- So our lambda function is created.
- And now you go on configuration
- And in configuration go general configuration section.
- And click on Edit.

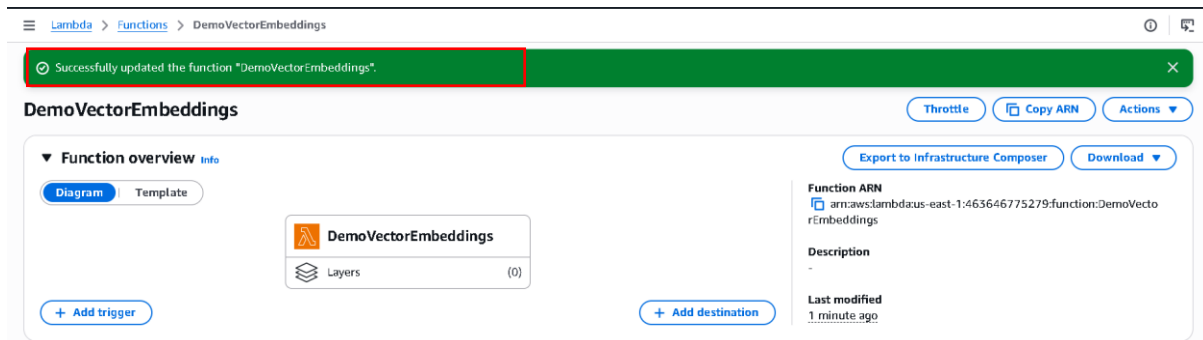


- And in configuration go general configuration section.
- And click on Edit.

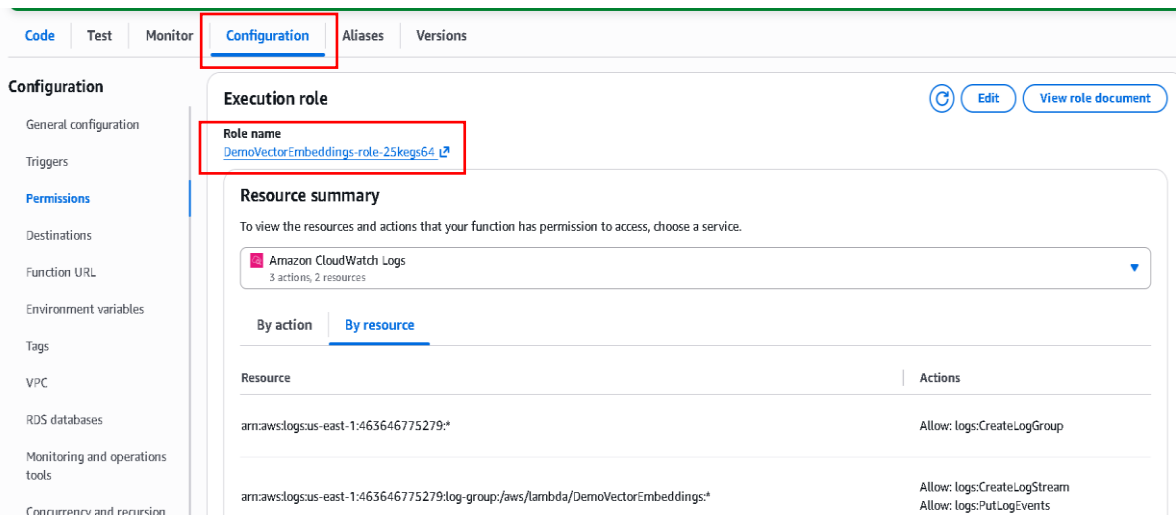


- The default timeout for lambda function is 3 second
- So I will make it 1 min 3 sec because it might have to do some processing.

- And click on save.



- Function is successfully updated.
- Now Lambda Function also requires access to Amazon Bedrock.



- So we'll give it the required permissions.
- So go to configuration and click on Permission
- Now click on role name
- And then in role name click on ADD Permissions

DemoVectorEmbeddings-role-25keys64 Info

Summary

Creation date
April 14, 2026, 13:02 (UTC+05:30)

Last activity
-

ARN
arn:aws:iam::463646775279:role/service-role/DemoVectorEmbeddings-role-25keys64

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1) Info

You can attach up to 10 managed policies.

Filter by Type
All types

Search

Policy name	Type	Attached entities
☒ AWSLambdaBasicExecutionRole-1151ccf...	Customer managed	1

Buttons: Simulate, Remove, Add permissions (dropdown: Attach policies, Create inline policy)

- In Add Permission click Attach Policies.

Current permissions policies (1)

Other permissions policies (1/2000)

Filter by Type
All types

Search

Policy name	Type	Description
☐ aal_geforces3limitedpolicy	Customer managed	-
☐ AccountManagementFromVercel	AWS managed	-
☒ AdministratorAccess	AWS managed - job function	-
☐ AdministratorAccess-Amplify	AWS managed	-
☐ AdministratorAccess-AWSClasticBeanstalk	AWS managed	-
☐ AffPolicy1	Customer managed	-
☐ AiDevOpsAgentAccessPolicy	AWS managed	-
☐ AiDevOpsAgentFullAccess	AWS managed	-
☐ AiDevOpsAgentReadOnlyAccess	AWS managed	-
☐ AiDevOpsOperatorAppAccessPolicy	AWS managed	-

- So we'll give admin access
- This is not done in production Environment but for this demo we'll just give it admin access.
- Now just scroll down click on add Permissions.

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- User groups
- Users
- Roles**
- Policies
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer

DemoVectorEmbeddings-role-25keys64 Info

Summary

Creation date
April 14, 2026, 13:02 (UTC+05:30)

Last activity
-

ARN
arn:aws:iam::463646775279:role/service-role/DemoVectorEmbeddings-role-25keys64

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) Info

You can attach up to 10 managed policies.

Buttons: Simulate, Remove, Add permissions

- Now Policy was successfully attached to role.
- Go back on lambda function tab and click on code.
- And paste the below code in the code section.

```

import boto3
import json

def lambda_handler(event, context):

    # ⚡ Step 1: Create Bedrock client
    client = boto3.client("bedrock-runtime", region_name="us-east-1")

    # ⚡ Step 2: Model ID (Titan Embeddings V2)
    model_id = "amazon.titan-embed-text-v2:0"

    # ⚡ Step 3: Get input text (dynamic or default)
    input_text = event.get("text", "What is the price of apple?")

    # ⚡ Step 4: Create request
    request_body = {
        "inputText": input_text
    }

    # ⚡ Step 5: Convert to JSON
    request = json.dumps(request_body)

    # ⚡ Step 6: Call Bedrock model
    response = client.invoke_model(
        modelId=model_id,
        body=request
    )

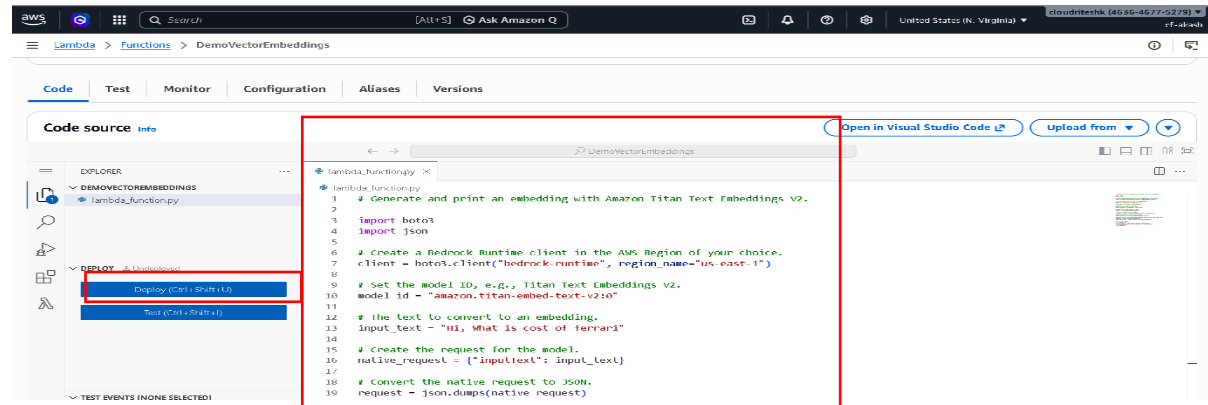
    # ⚡ Step 7: Read response
    response_body = json.loads(response["body"].read())

    embedding = response_body["embedding"]
    token_count = response_body["inputTextTokenCount"]

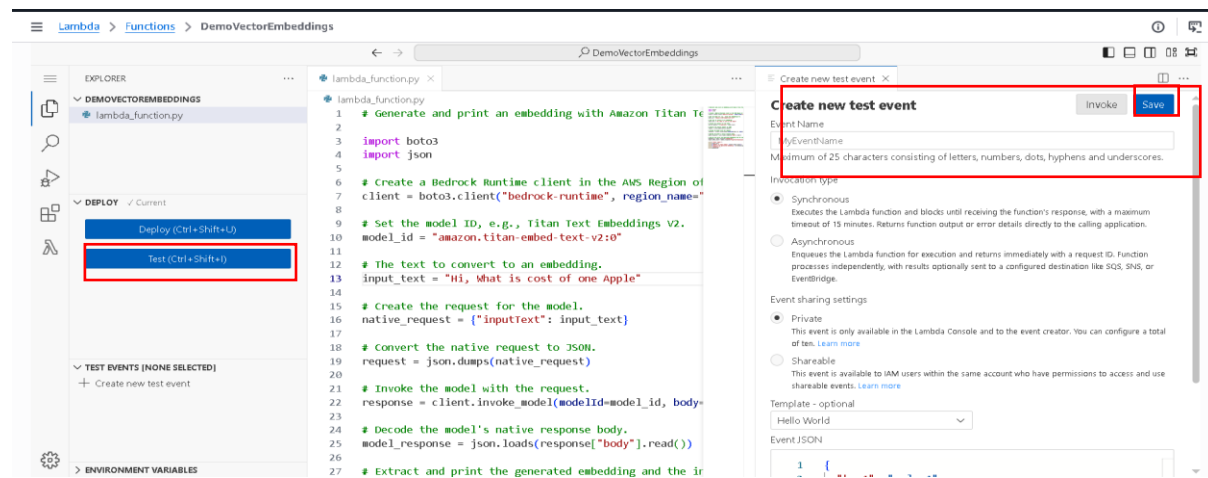
    # 🔥 Step 8: Print logs (CloudWatch)
    print("==== DEBUG LOGS =====")
    print("Input Text:", input_text)
    print("Token Count:", token_count)
    print("Embedding Size:", len(embedding))
    print("First 10 Embedding Values:", embedding[:10])

    # ⚡ Step 9: Return response (avoid full embedding)
    return {
        "statusCode": 200,
        "body": {
            "input": input_text,
            "token_count": token_count,
            "embedding_size": len(embedding),
            "sample_embedding": embedding[:10]    # only sample
        }
    }

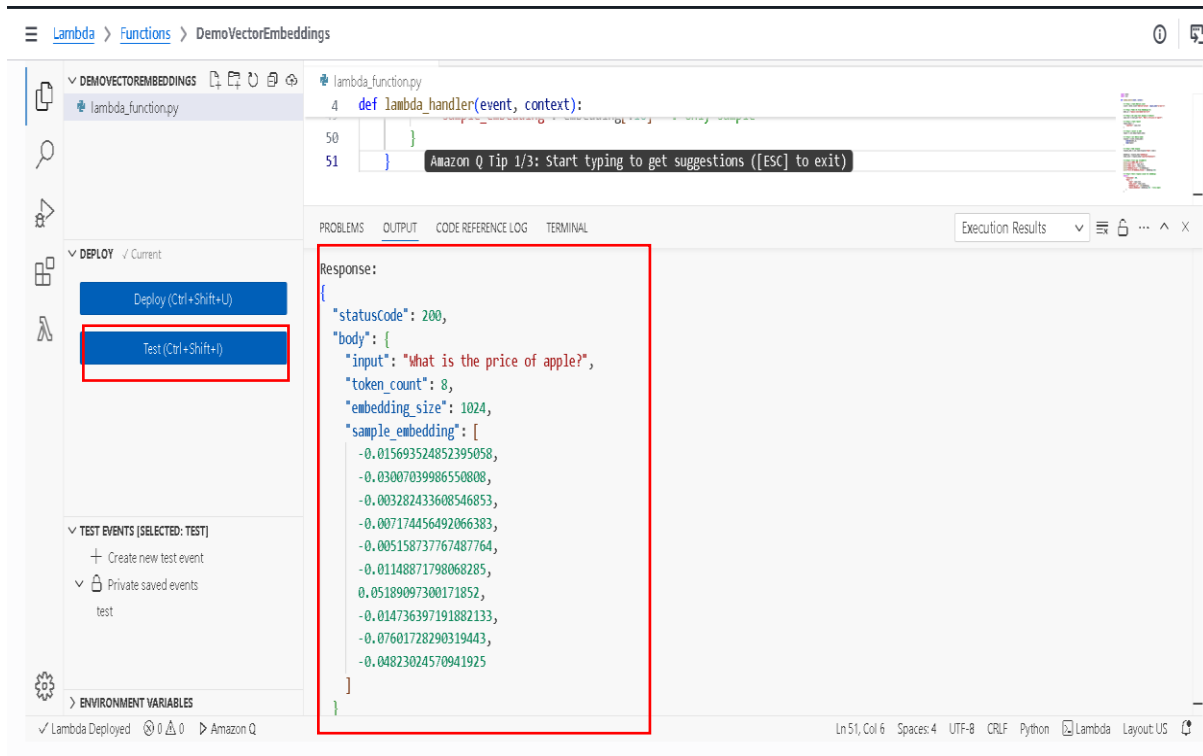
```

- Now click on Deploy
- Our code is deployed



- Now click on Test
- A popup window comes select create new test event
- Write event name (like test) and click on save.



- And then again click on test button.
- Your function runs and give output.